Singleton Session Beans

Chapter 7

Sample code updated to ee10:

https://github.com/bdlink/oreilly-ejb-6thedition-book-examples.git

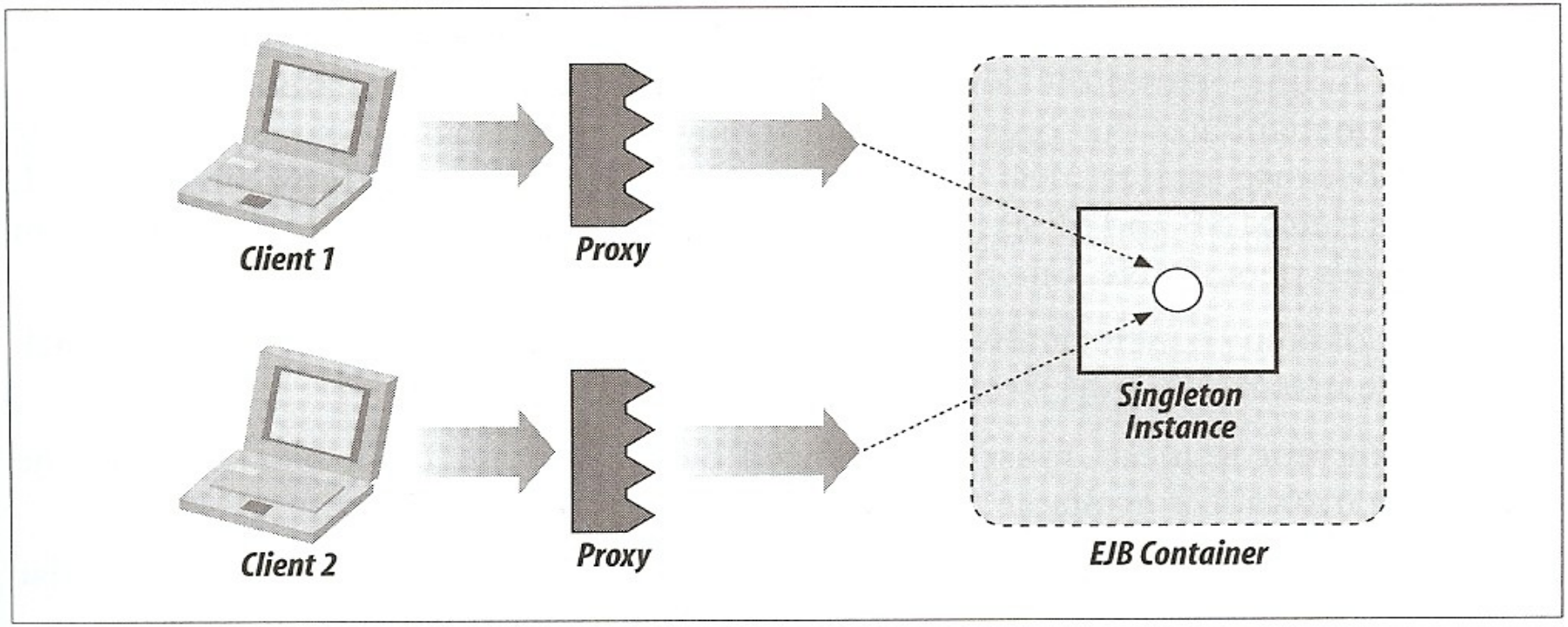
use ee10 branch

Introduction

- Have seen the isolation that stateless and stateful session beans provide to clients
  - Each bean method is called by a single client thread at a time
  - Bean developer does not worry about concurrency
- Sometimes need to provide a single shared instance for all clients
  - EJB 3.1 adds singleton session bean
  - `@jakarta.ejb.Singleton`
  - Simple conceptual model

One Backing Instance

- All invocations on singleton session bean use same backing instance
  - Multiple calls at same time

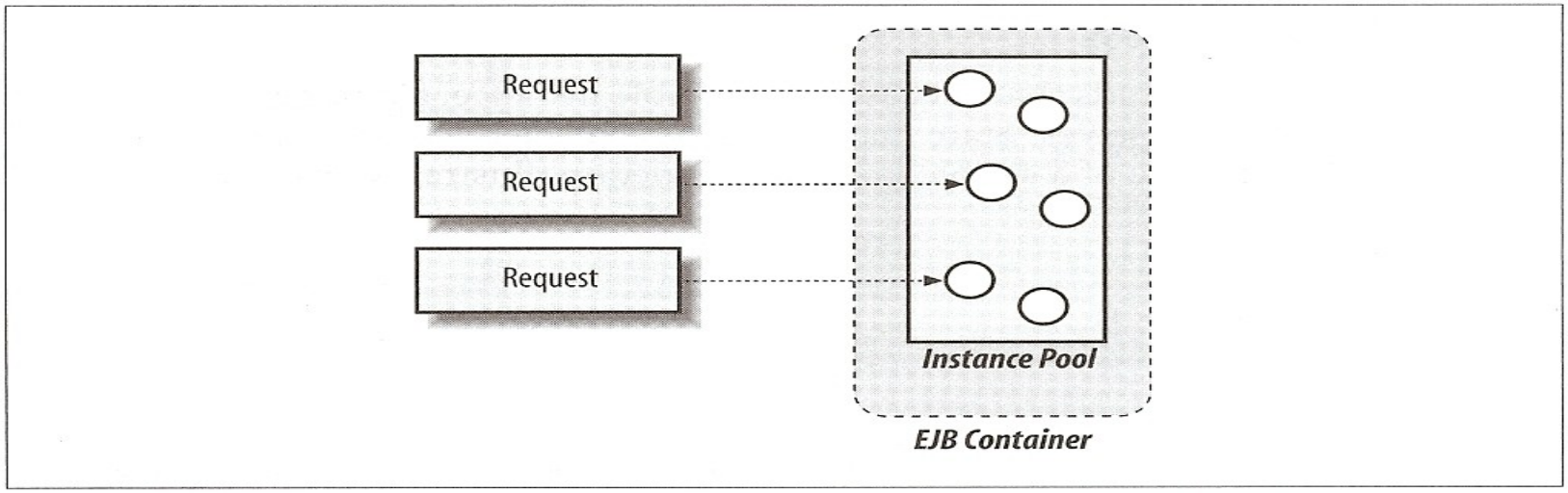


Singleton SB Implications

- Instance's state shared by all requests
- Any number of requests concurrent at any time
  - Singleton SB must be thread-safe
- Locking/synchronization to ensure thread safety may block new requests – bottleneck
  - May not show up in unit testing
  - May kill production performance
  - Need to design carefully
- Lowest memory footprint of session beans

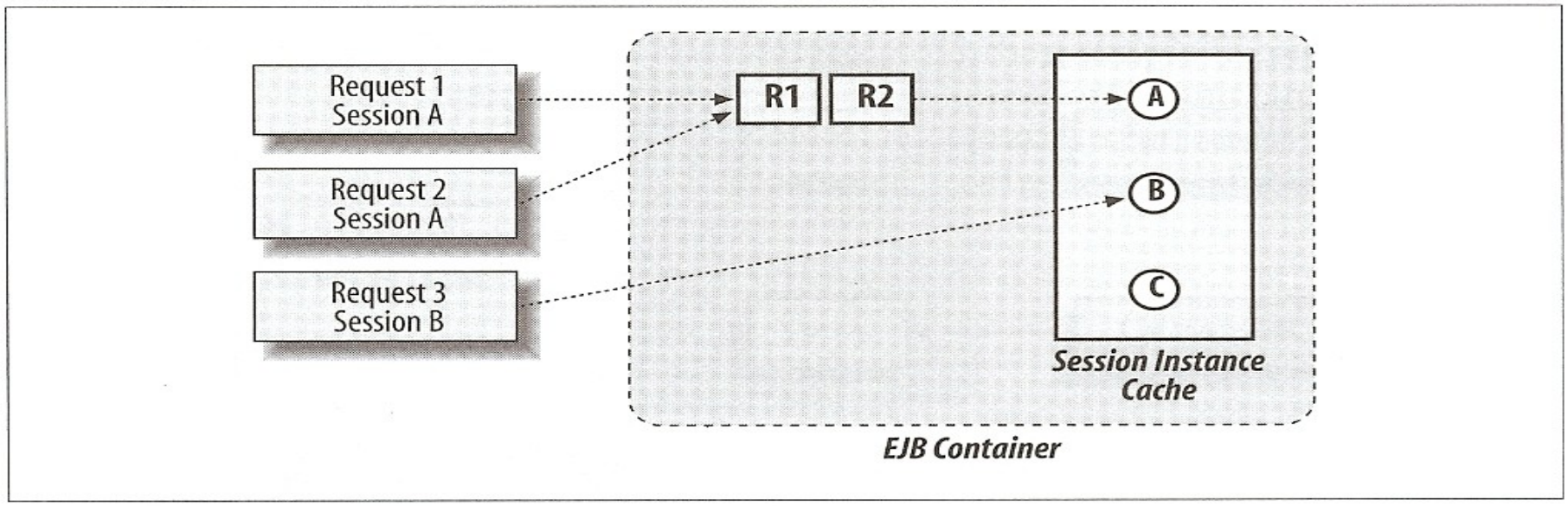
How is Concurrency Handled with EJBs?

- Have seen that for SLSB and SFSB only one request can access a bean instance at a time
  - Bean instance does not have to be thread-safe
- For stateless session beans each request goes to unique instance



Stateful Session Beans Queue

- Container serializes SFSB requests to same instance
  - Requests need to be queued by container
  - Infrequent occurrence

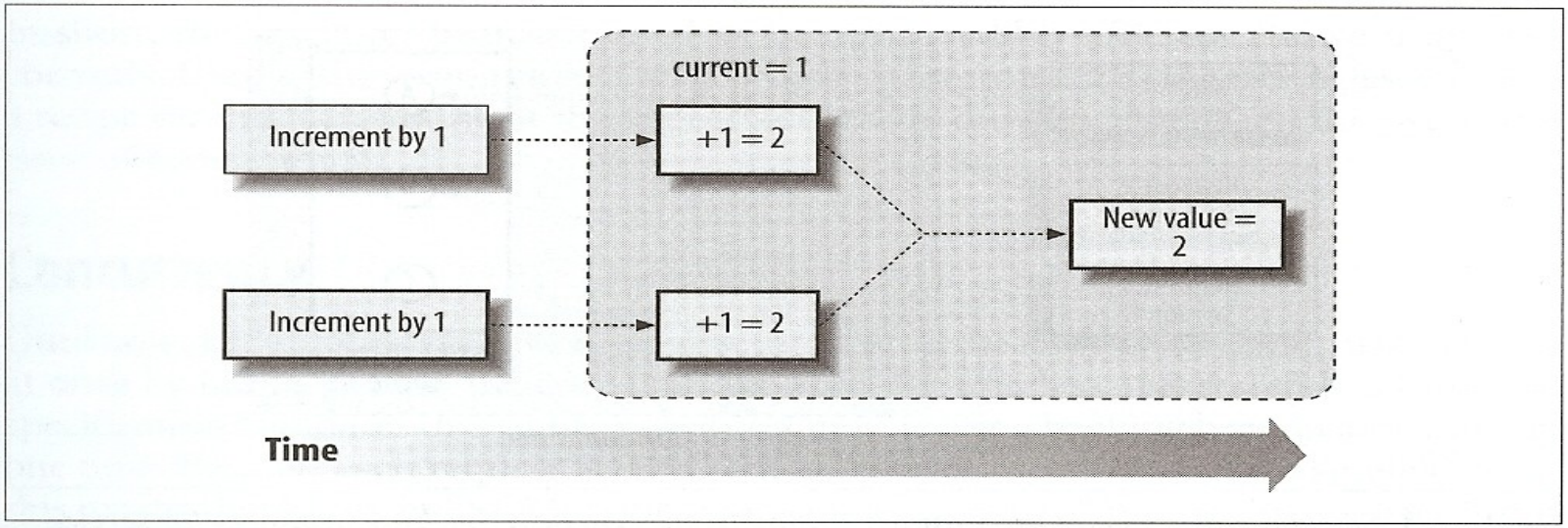


Cannot Prohibit Parallel SFSB Calls

- Text is mistaken on this point
- API for jakarta.ejb.ConcurencyManagement states
  - This annotation may be applied to stateful session beans, but doing so has no impact on the semantics of concurrency management for such beans.
  - The concurrency management type for bean-managed concurrency (BEAN) does not apply to stateful session beans.

Shared Mutable Access

- Any data accessed by multiple threads is a hazard

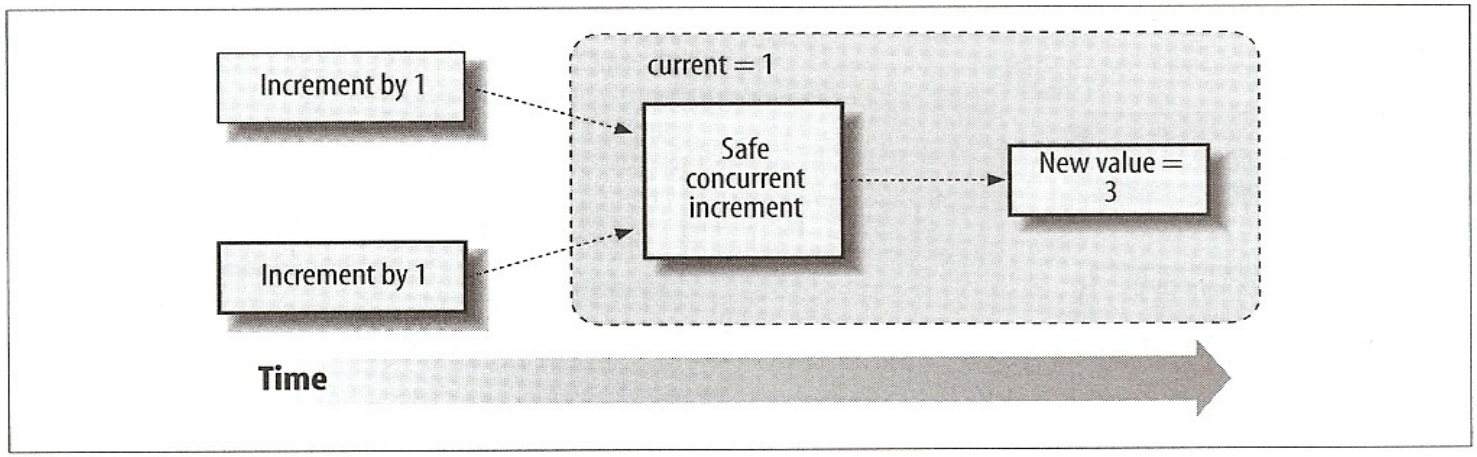


- java.util.concurrent has several data structures that are thread-safe
  - Various queues, maps, atomic primitives

Using AtomicInteger

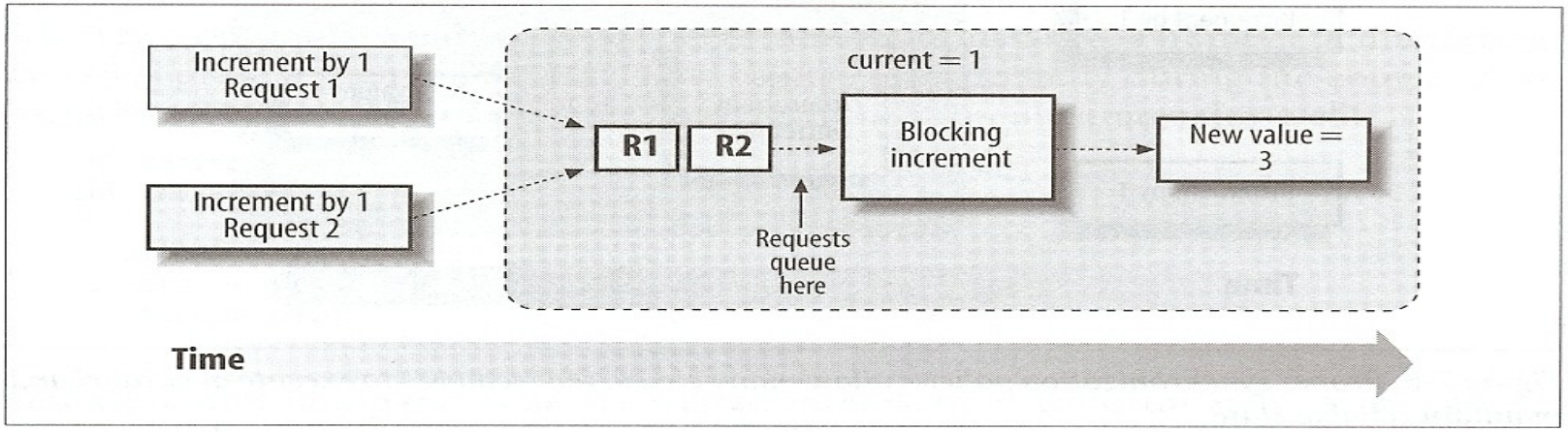
- For example, atomic operations for boolean, integer, long, references and arrays of these
  - In java.util.concurrent.atomic

```
AtomicInteger shared = new AtomicInteger(1);
public int increment() {
   return shared.incrementAndGet();
}
```



At Higher Level, Synchronize

- Using synchronized or locks can define critical sections that can be entered by one thread at a time
- Introduces a potential bottleneck so need to keep critical sections short



Container-Managed Concurrency

- Container will apply locking strategy
  - CMC is default for Singleton Session Beans
- Annotate class or methods with `@jakarta.ejb.Lock(`*`locktype`*`)`
  - locktype is enum defined in `jakarta.ejb.LockType`
  - READ allows concurrent access to all requests
    - As long as no write lock is currently held
  - WRITE (default) defines lock that can be held by one thread at a time
- Can specify timeout so calls do not block indefinitely
  - Use `@jakarta.ejb.AccessTimeout()`

Bean-Managed Concurrency

- To do it yourself, specify `@jakarta.ejb.ConcurrencyManagement(BEAN)`
- Responsibility of bean developer to make the bean thread-safe
  - Using locks, synchronized, classes from java.util.concurrent

Singleton SB Lifecycle

- Lifecycle similar to SLSB
  - Does Not Exist or Method-Ready
- Container can choose when to create bean instance
  - `Class.newInstance()`, injections, `@PostConstruct` method
  - Must be available before first method call
  - Lives in memory for life of application
- Can create at deployment time with annotation
  - `@jakarta.ejb.Startup`

Example: RSSCacheEJB

- Singleton SBs especially suitable for holding cache that is read-mostly
  - Parallel reads allowed in absence of writes
  - Can populate cache when application deploys
    - Before any requests
- Example defines an RSS caching service
  - Reads an RSS feed
  - Stores for fast access
- Updates temporarily block reads until complete

Value Objects (Data Transfer Objects)

- Define the RSS entries that will be sent to user
  - `RssEntry` interface with `String getAuthor, String getTitle, URL getUrl, String getDescription`
- Important issue: URL is mutable
  - If pass reference to internal state, local clients can modify using that reference
  - Make sure implementation makes a copy and passes that reference
  - Strings don't have that problem - immutable

Project ROME

- See http://rometools.org/
  - Open source framework for dealing with RSS feeds
- `RomeRssEntry` implements `RssEntry` using the ROME package
  - See text for code

RssCache Business Interface

- Returns read-only copy of RSS feed cache
- `refresh()` refreshes the cache from the feed

```
package org.jboss.ejb3.examples.ch07.rsscache.spi;

import java.net.URL;
import java.util.List;

public interface RssCacheCommonBusiness {

   List<RssEntry> getEntries();

   URL getUrl();

   void refresh();
}
```

RssCacheBean Implementation

- Singleton bean
  - Need to worry about multithreading
  - Uses container managed concurrency
- getEntries, getUrl are READ locked
  - anyone can read in parallel when cache is not being refreshed
- refresh is WRITE locked
  - Also @PostConstruct so cache is initialized at startup
- See code

Summary

- Singleton beans are useful when one instance of a bean is shared over all clients
- Only Session Bean type that needs to be thread safe
- Offers read/write locking container-based concurrency
  - Similar in concept to database locks
- Less commonly needed than other session bean types